SAMPLE QUESTIONS

1. **How Was your day**
2. **Why the deployment took time if you have all automation in place**

   Even with full automation in place, deployment can take longer than expected due to several factors. Here are some common reasons why deployment might be delayed despite automation:

   1. Infrastructure Issues

   Slow Provisioning of Cloud Resources – Sometimes, cloud providers (AWS, Azure, GCP) take time to spin up new instances, allocate storage, or set up network configurations.

   Scaling Bottlenecks – If auto-scaling is misconfigured, the system might take longer to provision new instances.

   2. Build & CI/CD Pipeline Delays

   Long Build Times – Large applications with heavy dependencies, complex code compilation, or container image creation can slow down the CI/CD pipeline.

   Parallel Job Failures – If multiple build/test jobs run in parallel and one fails, the whole process might need a restart.

   Artifact Storage Issues – Uploading/downloading large deployment packages from repositories like Docker Hub, Artifactory, or S3 can add delays.

   3. Testing Bottlenecks

   Integration & Regression Tests – Automated tests (e.g., Selenium, JUnit, PyTest) might take time, especially if test cases are extensive.

   Flaky Tests & Retrying – Some tests intermittently fail and require reruns, increasing deployment time.

   4. Configuration & Dependency Issues

   Environment-Specific Configurations – Differences in dev, staging, and production environments may cause unexpected issues.

   Third-Party API Delays – If external APIs (payment gateways, authentication services) respond slowly or fail, deployment can be blocked.

   5. Database Migrations & Schema Changes

   Large Data Migrations – If the deployment includes schema updates, large data migrations can take time.

   Locking Issues – Some database changes require table locks, causing performance degradation.

   6. Security & Compliance Checks

   Vulnerability Scans & Approvals – Automated security scans (e.g., SonarQube, Snyk, OWASP ZAP) might introduce delays if vulnerabilities are detected.

   Manual Approvals in CI/CD – Some organizations enforce manual approvals before pushing to production.

   7. Rollback & Blue-Green Deployment Issues

   Canary or Blue-Green Deployments – Routing traffic to new instances gradually can add extra time to deployment.

   Rollback Triggers – If an issue is detected post-deployment, automatic rollback processes can further delay the release.

   8. Network & Load Balancer Propagation

   DNS Propagation Delays – Changes to DNS records (e.g., updating CNAME or A records) can take time to propagate globally.

Load Balancer Health Checks – Some cloud providers take time to mark new instances as "healthy" before routing traffic.

9. Human Intervention

Approval Delays – If manual approvals are required at any stage, human intervention can slow down deployment.

Incident Handling – If a previous deployment caused issues, teams might take extra precautions before rolling out changes.

Conclusion

Even with automation, deployment delays can happen due to infrastructure, testing, database migrations, security checks, or manual interventions. Optimizing CI/CD pipelines, testing strategies, and infrastructure readiness can help reduce deployment time.

Manual Deployment

Manual deployment refers to the process of releasing and deploying an application or software update without using automated deployment tools or scripts. It typically involves human intervention at various stages, such as code compilation, configuration, transfer, and activation in the production environment.

Steps in Manual Deployment

A typical manual deployment process might include:

- **Code Preparation & Compilation**
  - Developers finalize and compile the code on a local machine.
  - The compiled code (binaries, artifacts) is manually packaged.
- **Transferring Files to the Server**
  - The packaged files are uploaded to the production server via SSH, FTP, or SCP.
  - Files might be extracted or placed in specific directories.
- **Database & Configuration Updates**
  - Any required database schema changes or updates are executed manually (e.g., running SQL scripts).
  - Configuration files (e.g., .env, config.yaml, nginx.conf) are manually edited and placed in the correct locations.
- **Restarting Services**
  - The application or web server (e.g., Apache, Nginx, Tomcat) is restarted manually.
  - If using containers, they may be manually stopped and restarted.
- **Testing & Verification**
  - Engineers manually check logs, monitor performance, and test endpoints to ensure everything is working.
- **Rollback (if necessary)**
  - If issues arise, a rollback is done by restoring previous versions manually.

When is Manual Deployment Used?

- Initial Setup – When setting up a new application for the first time.
- Small-Scale Applications – For projects that don't require frequent updates.
- Emergency Fixes – If an automated CI/CD pipeline fails and urgent deployment is needed.
- Controlled Releases – When an organization needs full control over the deployment process.

Challenges of Manual Deployment

❌ Time-Consuming – Requires multiple manual steps, increasing deployment time.
❌ Error-Prone – Human errors, such as misconfigurations or missing files, can cause failures.
❌ Inconsistent Deployments – Repeating the same process manually can lead to inconsistencies.
❌ Difficult Rollbacks – Undoing a failed deployment manually can be complex.

Alternative: Automated Deployment

To overcome these challenges, Continuous Integration/Continuous Deployment (CI/CD) tools (e.g., Jenkins, GitHub Actions, GitLab CI/CD, AWS CodeDeploy) can automate the deployment process, ensuring faster, error-free, and consistent releases.

3. How can you minimize the deployment time?

To minimize deployment time, you can optimize various aspects of the CI/CD pipeline and infrastructure. Here are some key strategies:

1. Optimize CI/CD Pipeline

- Parallel Execution: Run tests, builds, and deployments in parallel instead of sequentially to reduce overall time.
- Incremental Builds & Caching: Use incremental builds and cache dependencies, containers, or compiled files to avoid rebuilding everything from scratch.
- Optimize Tests: Run unit tests first and only trigger integration and end-to-end tests when necessary. Prioritize critical test cases.
- Reduce Approval Delays: Automate approvals where possible or use pre-approved release candidates for faster rollouts.

2. Optimize Infrastructure

- Pre-Provision Resources: Keep infrastructure ready and use blue-green or canary deployments to minimize downtime.
- Use Infrastructure as Code (IaC): Automate infrastructure setup with Terraform, Ansible, or CloudFormation to ensure consistent and rapid deployment.
- Containerization: Deploy applications as lightweight Docker containers to eliminate environment inconsistencies.

3. Optimize Deployment Strategy

- Rolling Deployments: Deploy in stages instead of restarting all services at once.
- Blue-Green Deployment: Keep two versions live and switch traffic to the new one instantly.
- Canary Releases: Deploy updates to a small group before rolling out to all users, minimizing risk and rollback time.
- Zero-Downtime Deployments: Use feature flags and database migrations that don't require downtime.

4. Optimize Database Operations

- Database Connection Pooling: Avoid excessive connection creation by using connection pooling techniques.
- Optimize Database Migrations: Use non-blocking, incremental schema updates instead of large schema changes.

5. Optimize Monitoring & Rollbacks

- Automated Health Checks: Implement automated checks to quickly detect issues and rollback if needed.
- Faster Rollbacks: Maintain previous versions of code and configurations to enable quick rollbacks in case of failure.

4. **Explain your last project application's architecture in high level**
5. **What type of web, app and DB servers used in your last project?**
6. **Have you encountered any P1 issue give some example and how did you handled it**
7. **What is mean by SLA, SLO, SLI**

SLA, SLO, and SLI are key terms used in service reliability and performance monitoring. Here's what they mean:

1. Service Level Agreement (SLA)

An SLA is a formal contract between a service provider and a customer that defines the expected level of service.
It includes commitments like uptime, response time, and support availability.
Example: "The service will have 99.9% uptime per month, or the provider will issue a refund."

2. Service Level Objective (SLO)

An SLO is an internal target set by the service provider to maintain service quality.
It's usually stricter than an SLA and helps in monitoring performance proactively.
Example: "The target uptime for our service is 99.95% per month to ensure we meet our 99.9% SLA commitment."

3. Service Level Indicator (SLI)

An SLI is a metric that measures the actual performance of a service against an SLO.
It tracks parameters like latency, error rate, and availability.
Example: "The actual uptime recorded last month was 99.92%, meeting our 99.9% SLA but slightly below our 99.95% SLO."

Relationship Between SLA, SLO, and SLI

📌 SLIs help measure service performance.
📌 SLOs define the target performance.
📌 SLAs are the promises made to customers based on those targets.

8. **How you identified the toils and what action you have taken**

Identifying Toil and Actions Taken to Reduce It

🔍 What is Toil?

Toil refers to repetitive, manual, and time-consuming tasks in operations that do not add long-term value. It often leads to inefficiencies and prevents teams from focusing on innovation.

🛠️ How I Identified Toil

I used the following methods to identify toil in the deployment process:

📊 Analyzing Repetitive Tasks

Noticed frequent manual approvals delaying deployments.
Found engineers spending excessive time on log analysis and incident handling.

⏳ Measuring Time Spent

Monitored deployment and rollback times, finding slow database migrations and environment provisioning.

📝 Tracking Human Intervention

Noticed frequent manual script execution for server restarts and configuration updates.

🔍 Reviewing Incident Reports

Identified recurring deployment failures due to misconfigurations and lack of rollback automation.

📢 Gathering Team Feedback

Developers reported slow CI/CD pipelines, requiring manual intervention to trigger tests and deployments.

🚀 Actions Taken to Reduce Toil

Identified Toil	Action Taken
Manual approvals delaying releases	Implemented automated approvals for non-critical changes.
Frequent log analysis and debugging	Integrated centralized logging and monitoring (ELK, Prometheus, Grafana) for faster debugging.
Slow database migrations	Used non-blocking schema updates and pre-migration validation to minimize downtime.
Manual deployment rollbacks	Automated blue-green and canary deployments for safe rollbacks.
Infrastructure provisioning delays	Adopted Infrastructure as Code (Terraform, Ansible) for automated provisioning.
Manual test execution	Shifted to automated testing (unit, integration, and end-to-end tests).
Slow CI/CD pipelines	Optimized pipelines by parallelizing test execution and caching dependencies.

📊 Results & Impact

- ☑ Deployment time reduced by 40%
- ☑ Incidents due to misconfigurations dropped significantly
- ☑ More focus on innovation instead of repetitive operational tasks

9. **What type of dashboard you have created in SPLUNK?**
10. **What are the metrics you consider for setting up monitoring for a new application on boarding?**
11. **Which monitoring you choose to set up monitoring for a new application SPLUNK, Dynatrace**
12. **How do you identify the fake alert in Dynatrace?**
13. **What is observability and telemetric data**
14. **What do you mean by leadership?**
15. **What kind of report you generate in your project on weekly or monthly basis to showcase the management?**
16. **How do you remediate the vulnerabilities?**
17. **SQL query to list rows matching a condition (listing the number of transactions happened for a month in)**

18. What is major difference between SNOW and BMC remedy

Major Differences Between ServiceNow (SNOW) and BMC Remedy

ServiceNow (SNOW) and BMC Remedy are both IT Service Management (ITSM) platforms used for incident management, problem management, change management, and asset management. However, they differ in several key areas:

Feature	ServiceNow (SNOW)	BMC Remedy
Deployment Model	Cloud-based SaaS (Software as a Service).	On-premises & cloud options available.
Ease of Use	Modern, user-friendly UI with drag-and-drop workflows.	Traditional UI, requires more customization.
Customization & Flexibility	Highly customizable with low-code/no-code tools and automation.	More rigid, requires developer expertise for customization.
Performance & Scalability	Scales easily with cloud infrastructure.	Can be complex to scale in on-prem setups.
ITSM Modules	Offers built-in ITSM and additional AI-driven automation tools.	Strong ITSM capabilities but requires additional modules for advanced automation.
Integration	Strong API support, integrates well with third-party tools (Jira, Slack, etc.).	Requires more configuration for third-party integrations.
Implementation Time	Faster implementation with predefined workflows.	Requires longer setup time due to heavy customization.
Cost	Subscription-based, costs depend on features used.	Higher upfront cost, especially for on-prem deployments.
AI & Automation	Uses AI-driven Virtual Agents and predictive intelligence.	AI capabilities exist but are less advanced compared to SNOW.
Market Preference	Popular for enterprises looking for cloud-first, modern ITSM.	Preferred by organizations needing deep customization and legacy system support.

🚀 Summary

Use ServiceNow (SNOW) if: You prefer a cloud-based, scalable, and modern ITSM tool with AI automation.

Use BMC Remedy if: You need a highly customizable on-prem solution with strong legacy system support.

19. Have you created any dashboard in SNOW?

20. What is business impacting issue. How did you handle those issue?

Major incident guidelines (MIM)

21. Do you interact with the end user to resolve the issue?

22. What is incident, problem and change process

Incident, Problem, and Change Management Process

These are key processes in IT Service Management (ITSM), often managed using ITIL (Information Technology Infrastructure Library) best practices.

🔵 Incident Management

📌 What is an Incident?

An incident is an unplanned disruption or reduction in the quality of a service. The goal is to restore normal service as quickly as possible.

Example:

A web application goes down unexpectedly.
Users cannot log in due to an authentication failure.

🛠️ Incident Management Process:

Detection & Logging – Identify and record the incident.
Categorization & Prioritization – Assign a severity level (Critical, High, Medium, Low).
Investigation & Diagnosis – Find the root cause.
Resolution & Recovery – Apply a fix or workaround.
Closure – Verify resolution and close the incident.

🎯 Goal: Minimize downtime and restore service ASAP.

🔵 Problem Management

📌 What is a Problem?

A problem is the underlying cause of one or more incidents. Unlike incident management (which focuses on quick fixes), problem management aims for permanent solutions.

Example:

Frequent server crashes → Investigation finds memory leaks in the application.
Repeated database failures → Identified as poor indexing strategy.

🛠️ Problem Management Process:

Problem Identification – Analyze recurring incidents.
Root Cause Analysis (RCA) – Use methods like 5 Whys, Fishbone Diagram, or Fault Tree Analysis.
Workaround or Permanent Fix – Provide a temporary fix (workaround) or implement a long-term solution.
Implementation & Review – Deploy the fix, monitor, and prevent recurrence.

🎯 Goal: Prevent future incidents by fixing the root cause.

🔵 Change Management

📌 What is a Change?

A change is any modification to the IT environment that can impact services. It must be controlled to minimize risks.

Example:

Deploying a new version of an application.
Migrating a database to a new server.

🛠️ Change Management Process:

Change Request Submission – Document the proposed change (who, what, why, impact).
Risk & Impact Assessment – Evaluate potential risks and dependencies.
Approval Process – Get approval from a Change Advisory Board (CAB) if required.
Implementation – Execute the change with rollback plans in place.
Post-Implementation Review – Verify success and document lessons learned.

🎯 Goal: Ensure smooth and risk-free changes to IT systems.

🔄 Relationship Between Incident, Problem, and Change

Incident → Temporary Fix (Quick resolution to restore service).
Problem → Root Cause Fix (Identify & eliminate recurring issues).
Change → Controlled Updates (Implement permanent improvements).

23. What is the biggest challenge in your current project?

Sample Questions

1. **How Was your day**
2. **Why the deployment took time if you have all automation in place**
3. **How can you minimize the deployment time**
4. **Explain your last project application's architecture in high level**
5. **What type of web, app and DB servers used in your last project?**
   Web – apache
   DB – mysql
6. **Have you encountered any P1 issue give some example and how did you handled it**
7. **What is mean by SLA, SLO, SLI**
8. **How you identified the toils and what action you have taken**
9. **What type of dashboard you have created in SLUNK**
10. **What are the metrics you consider for setting up monitoring for a new application on boarding**
11. **Which monitoring you choose to set up monitoring for a new application splunk, Dynatrace**
12. **How do you identify the fake alert in Dynatrace**
13. **What is observility and telemetric data**
14. **What do you mean by leadership**
15. **What kind of report you generate in your project on weeking or monthly basis to showcase the management**
16. **How do you remediate the vulnerabilities**
17. **SQL query to list rows matching a condition(listing the number of transaction happened for a month in)**
18. **What is major difference between SNOW and BMC remedy**
19. **Have you created any dashboard in SNOW**
20. **What is business impacting issue . How did you handle those issue**
21. **Do you interact with the end user to resolve the issue**
22. **What is incident, problem and change process**
23. **What is the biggest challenge in your current project**

24. **Explain your last project application's architecture in high level**

NAISS

It is a system, which collects and processes the intrusions occurred, communication received from TETRA radios, info related gate automation system

Up Stream: In to the system – BEL DBMS

Down Stream: From the NAISS system - NAISS Security Intelligence Hub

Upstream (Into the System – BEL DBMS)

Responsibilities:

Intrusion Data Collection – Captures and processes intrusion events from security sensors.

TETRA Radio Communication – Receives and logs communication data from TETRA radios.

Gate Automation System Information – Collects entry/exit data and automation system logs.

Data Validation & Storage – Ensures data integrity before storing it in the BEL DBMS.

Intermediate System – Threat Detection System (TDS)

Responsibilities:

Anomaly Detection – Identifies suspicious activity patterns from collected data.

Real-Time Threat Analysis – Processes data using AI/ML or rule-based threat detection.

Risk Classification – Categorizes threats based on severity (low, medium, high).

Alert Generation – Notifies security teams and triggers automated responses.

Data Enrichment – Enhances raw data with context before forwarding to NAISS.

Downstream (From the NAISS System – NAISS Security Intelligence Hub (NSIH))

Responsibilities:

Incident Alerts & Notifications – Sends real-time alerts based on threat assessment.

Security Dashboard Updates – Provides analyzed data for monitoring and decision-making.

Automated Response Triggers – Integrates with gate automation and security systems.

Reporting & Analytics – Generates reports for further security assessment and auditing.

1. Have you encountered any P1 issue give some example and how did you handled it

Real time incident:

As we are aware when we notice an incident, first we have to get the justification from the user.

After getting the justification, we have to pri

Yes I have encountered such incidents, first I ll communicate with all my stake holders with clear and concise information so that non-technical ppl also understands the situation.

First ill check if anything is possible to solve from my end, if it not possible ill reach out to concise team. I was continuously following with them till the incident get resolved, meanwhile ill communicate with all the stake holders regarding the progress of the incident so tht every one are aware of the situation.

Later I had reached to

When we get a P1 incident,

Assignments,

1. List all files
2. Specific files
3. Larger files
4. List processes
5. Kill processes
6. Start, stop and restart processes
7. Change owner and group permissions

8. Grep statements
9. Sql languages
10. Joins
11. Delete, insert, truncate, select
12. Unions